as in rspec --formatter doc. But it would be nicer not to require this extra step, and instead just use RSpec's default formatter if none is passed in.

The configure block we have shown here would prevent RSpec from providing a default formatter. Instead, we need to wait until the formatter has been set, and *then* add ours. To do so, we have put our initialization code in a before(:suite) hook:

09-configuring-rspec/02/configuring_rspec/rspec/print_failures_eagerly.rb
```
RSpec.configure do |config|
  config.before(:suite) do
    config.add_formatter RSpec::PrintFailuresEagerly::Formatter
  end
end
```

This library needs to take care of one last detail: cleaning up the output—specifically, by preventing failures from being printed twice. In the next section, we're going to see how to do that.

Cleaning Up the Output

RSpec formatters typically print a list of failures at the end of their output. Since we've already shown the failure messages during the RSpec run, we don't need to show them a second time.

Both the progress and documentation formatters listen for the dump_failures formatter event that RSpec sends at the end of a test run. They react to this notification by printing a list of failures. This behavior lives in a common dump_failures method in the BaseTextFormatter class inherited by the two formatters.

We've defined our own version of this method that prints nothing so that we don't get duplicate failure messages. This new dump_failures method goes into a module called SilenceDumpFailures, which we can then prepend onto the RSpec formatter base class to get rid of the extra output:

09-configuring-rspec/02/configuring_rspec/rspec/print_failures_eagerly.rb
```
module SilenceDumpFailures
  def dump_failures(_notification)
  end

  RSpec::Core::Formatters::BaseTextFormatter.prepend(self)
end
```

To override the formatters' behavior, we needed to monkey-patch RSpec—that is, change its behavior by opening up a core RSpec class and modifying